[INSERT UNIVERSITY LOGO HERE]

DEPARTMENT OF COMPUTER SCIENCE

Faculty of Information Technology

INDIVIDUAL ASSIGNMENT REPORT

Domain Modeling & Abstraction

UNIVERSITY COURSE REGISTRATION SYSTEM

STUDENT SUBMISSION DETAILS

Student Name:	[Your Full Name]
Student ID / Registration #:	[Your Roll / ID Number]
Course Name & Code:	Software Design & Architecture (CS-302)
Instructor Name:	[Instructor's Name]
Submission Date:	[Submission Date]

Domain Model & Abstraction Report

SOFTWARE DESIGN & ARCHITECTURE — ASSIGNMENT 01

Format: Short Report (1–2 Pages)
Domain: University Registration

1. DOMAIN SELECTION

Selected Domain: University Course Registration System

Brief Context: This domain handles academic course enrollment for university students each semester. It manages course catalogs, term offerings, prerequisite dependencies, student registrations, class capacities, and tuition fee calculations.

2. CANDIDATE CONCEPTUAL CLASSES (10 IDENTIFIED)

CONCEPTUAL CLASS	ONE-LINE JUSTIFICATION
Student	Represents an individual enrolled at the university who registers for courses and views schedules.
Course	Represents an academic subject in the catalog (e.g., CS101) with code, description, and credits.
CourseSection	Represents a specific term offering of a course with dedicated schedule, room, instructor, and seats.
Instructor	Represents a faculty member assigned to teach one or more course sections.
RegistrationRecord	Associates a student to a section, capturing enrollment status, timestamp, and final grade.
Prerequisite	Defines academic course dependencies required before enrolling in advanced sections.
Waitlist	Manages an ordered queue of eligible students awaiting seat availability in full sections.
Department	Represents an academic division managing faculty allocations and course offerings.
Schedule	Represents the aggregated weekly timetable layout for a student or instructor.
TuitionBill	Calculates financial assessment breakdown based on total enrolled credit hours.

3. DETAILED SPECIFICATIONS FOR 3 SELECTED CLASSES

`class Student`

ATTRIBUTES

- `studentID: String`
- `fullName: String`
- `email: String`
- `completedCredits: Integer`

METHODS / OPERATIONS

- `registerForCourse(sectionID): Boolean`
- `dropCourse(sectionID): Boolean`
- `getCompletedCourses(): List<Course>`

`class CourseSection`

ATTRIBUTES

- `sectionID: String`
- `capacity: Integer`
- `enrolledCount: Integer`
- `classroom: String`

METHODS / OPERATIONS

- `isFull(): Boolean`
- `addStudent(studentID): Boolean`
- `getAvailableSeats(): Integer`

`class RegistrationRecord`

ATTRIBUTES

- `registrationID: String`
- `registrationDate: Date`
- `status: EnrollmentStatus`
- `finalGrade: String`

METHODS / OPERATIONS

- `updateStatus(newStatus): Void`
- `recordGrade(grade): Void`
- `calculateFees(): Double`

4. MAIN ABSTRACTION DECISIONS

Explanation of Omitted Details: In modeling this domain, several peripheral university functions were deliberately left out, such as instructor payroll processing, campus housing and dining accounts, and facility maintenance tracking. These elements are non-essential to the primary course registration workflow. Excluding them keeps the domain model focused on core enrollment validation, capacity enforcement, and prerequisite checking without introducing unnecessary complexity.

5. INVENTED SOFTWARE OBJECT (PURE FABRICATION)

Invented Object: PrerequisiteValidator

Purpose: `PrerequisiteValidator` is a pure fabrication object (a domain service) designed to maintain high cohesion and low coupling. Rather than placing complex requirement-checking logic inside the `Student` or `CourseSection` classes, this service evaluates a student's transcript against course prerequisites to determine enrollment eligibility safely and independently.